A Graph-Native Coding Agent for Code-Impact Analysis

Abstract

Large-language-model (LLM) coding agents increasingly rely on auxiliary code-analysis tools to navigate large repositories, most commonly by attaching a static-analysis backend through the Model Context Protocol (MCP). We argue that bolting a code graph onto an agent as an optional tool under-uses it: the agent must both *decide* to query the graph and *rank* its raw output mid-reasoning, which it does inconsistently. We instead present a *graph-native* agent in which the program-dependency graph is the agent's primary retrieval surface, queried before the first generated token and ranked into a precise, budget-bounded shortlist by structural signal. We study this design on *code-impact analysis*—predicting the blast radius of a method change—using a benchmark of fourteen tasks derived from real merged pull requests on Apache Hadoop (12,490 Java files), with a held-out split, deduplication, a budget-capped F1 metric, and a static-analysis oracle as a hard floor. Holding the model, repository, and prompt fixed across arms, the graph-native agent attains a mean impact F1 of **0.70**, versus 0.61 for the same model with an MCP code graph and 0.57 for a tool-free baseline, while emitting the fewest output tokens (≈3.0k vs. 3.7k and 5.5k). The ranking component alone improves over the raw static blast radius from F1 0.17 to 0.52 on held-out tasks and beats the oracle floor on all of them. We find that no single arm dominates every task, which we take as evidence the benchmark is discriminative rather than saturated, and we are explicit about the central limitation: these tasks measure retrieval, not code synthesis.

1 Introduction

A coding agent that is about to modify a function must first understand what depends on it. Getting this *impact analysis* wrong is a common failure mode: a change that looks local breaks a caller three packages away. Human engineers answer the question with IDE "find-usages" and call hierarchies—features backed by a program-dependency graph. LLM agents, lacking that backing by default, fall back on lexical search: they grep for the symbol name and read files until they are reasonably confident. On a repository where a symbol is referenced thousands of times, this is both expensive and unreliable.

The prevailing remedy is to expose a static-analysis backend to the agent as a callable tool, typically over the Model Context Protocol (MCP). This helps, but it leaves two decisions to the model that it is poorly suited to make. First, the agent must *choose* to consult the graph rather than grep—a choice it makes inconsistently. Second, and more importantly, the raw output of an impact query is a *firehose*: a reverse-reachability query over a large codebase returns hundreds of files with near-perfect recall but very low precision, listed in source order. The agent is then expected to rank and filter this list *in context*, while also reasoning about the task—a step it performs unevenly, sometimes transcribing the whole list and sometimes discarding most of it.

We propose moving retrieval and ranking out of the model and into the harness. Our system, GraphNative, treats the dependency graph as the agent's *primary* retrieval surface rather than an optional tool. Concretely, it makes three design commitments:

(C1) **Graph-first retrieval.** The graph is queried *before* the agent's first token; the agent begins with the relevant structure already in context, rather than deciding whether to fetch it.

(C2) **Rank, don't dump.** The raw blast radius is re-ranked by structural graph signal into a small, budget-bounded shortlist that is mostly correct, so the model is not asked to rank a firehose mid-reasoning.

(C3) **Draft, then refine.** The agent is handed a pre-drafted high-confidence answer to *refine* with the graph tools, rather than a flat list to reason over from scratch.

Contributions. (1) A graph-native agent architecture for code-impact analysis that injects a ranked dependency shortlist at turn 0 and frames it as a draft to refine (Sec. 4). (2) A structural ranking function that converts a high-recall/low-precision static blast radius into a precise budget-bounded shortlist using only graph-topology and naming signals, with no access to ground truth

(Sec. 4.2). (3) A gaming-resistant benchmark built from real merged pull requests, with a held-out split, deduplication, a budget-capped metric, and a static-analysis oracle floor (Sec. 5). (4) A controlled three-arm evaluation isolating the harness as the only variable, showing the graph-native design is simultaneously more accurate and cheaper than an MCP-tool baseline, together with an honest account of where it does *not* win and what the results do and do not establish (Sec. 6).

2 Related Work

Tool-augmented coding agents. Modern agents extend an LLM with retrieval and execution tools; the Model Context Protocol has become a common substrate for attaching such tools, including code-analysis backends. In this paradigm the tool is reactive—invoked at the model's discretion and returning raw results. Our work keeps the same underlying graph backend but inverts the control flow: retrieval is proactive (turn-0) and its output is ranked by the harness before the model sees it.

Program-dependency graphs and impact analysis. Reverse-reachability over call/reference graphs is the classical basis for change-impact analysis in software engineering. Such analyses are sound-leaning—they favor recall, returning every potentially affected element—which is precisely what makes their raw output unsuitable as a final answer under a precision-sensitive metric. We treat the static blast radius not as the answer but as a high-recall candidate set to be re-ranked, and we use it (scored identically to the agents) as an oracle floor.

Retrieval ranking for code. Retrieval-augmented systems typically rank candidates by lexical or embedding similarity. Our ranker is deliberately different: it ranks by *structural* signals from the dependency graph (reference density, one-hop caller adjacency, type-name inheritance conventions, package locality), which are cheap, deterministic, and directly aligned with what "affected by a change" means.

3 Problem Definition

Let a repository be a set of files. A change is specified by a *subject symbol* s (the "anchor"; e.g. a method or class about to change) defined in a subject file. The code-impact-analysis task is to return an ordered list of files that genuinely depend on s and could be affected by the change. Ground truth is the set G of files actually modified alongside s in a real fix.

A system's prediction is evaluated as a ranked list truncated to a budget of $k = 20$ files. With $\hat{G}$ the truncated prediction (matched to G on file basename), we report precision $|\hat{G} \cap G|/|\hat{G}|$, recall $|\hat{G} \cap G|/|G|$, and their harmonic mean F_1. The budget cap is essential: without it, a system could maximize recall by returning the entire reverse-reachability set, which is uninformative. F_1 at a fixed budget rewards getting the *right few* files, not the most.

4 Method

GraphNative runs a standard headless coding-agent loop, with one architectural change: a code-graph retrieval-and-ranking stage executes before the agent's first turn, and its output is injected into the initial context. The graph backend exposes the usual queries (callers, callees, node lookup, search, and reverse-reachability "impact"); the agent retains access to all of them to verify and adjust. The novelty is what the harness does at turn 0.

4.1 Turn-0 retrieval

For the anchor s, the harness issues two queries:

- **Impact set** $I(s)$: reverse-reachability to depth 2. High recall, low precision; hundreds of files in source order.
- **Caller set** $C(s)$: the one-hop direct callers. High precision *when* it covers the answer, but possibly disjoint from G for widely-used types.

4.2 Structural ranking

Each candidate file f (the union of files in $I(s)$ and $C(s)$, excluding the subject's own file) is scored from four signals, none of which inspect ground truth:

- *Reference density* $\text{refs}(f)$: the number of affected symbols residing in f. The primary, uncapped signal—a file that uses the changing symbol in many places is more likely affected.
- *Direct-caller adjacency* $[f \in C(s)]$: the highest-precision one-hop signal, added as a bounded *bonus* rather than imposed as a dominating tier.

A direct caller that is also dense rises to the top, but a single low-density caller does not leapfrog a far denser dependent. This matters when $C(s)$ is *disjoint* from G: a hard caller-first tier would then bury the true dependents beneath many wrong callers.

- *Type-name match*: f's basename contains the anchor's type word (e.g. `MonotonicClock` for `Clock`, `NameNodeHttpServer` for `HttpServer2`). By naming convention these are implementors or subclasses—affected by a change to the base type, yet often appearing only in $I(s)$ at low density, so no other signal surfaces them. A minimum word length prevents short anchors from over-firing.
- *Package locality*: f shares the subject's package; a small tie-break for co-located collaborators.

Test-file demotion. On a real codebase, 40–55% of the files an impact query returns are test sources (e.g. `Test*`, `*Test`, `Mock*`, or anything under a `/test/` root), and they frequently exhibit the *highest* reference density. Yet the files changed in a production fix are production code; test files are essentially never in G. They are therefore demoted below every real candidate.

Scoring. Signals combine additively with a large test penalty:

$$\begin{aligned}\text{score}(f) = \text{refs}(f) &+ w_{\text{dir}}\,[f \in C(s)] \\ &+ w_{\text{name}}\,[\text{name}] + w_{\text{pkg}}\,[\text{pkg}] \\ &- w_{\text{test}}\,[\text{test}],\end{aligned}$$

with w_{dir}=8, w_{name}=10, w_{pkg}=4, w_{test}=10^5; files are sorted by score, ties broken by density then name. Weights reflect the structural role of each signal rather than a fit to the evaluation: held-out F1 varies by under 0.025 as w_{name} ranges over $[5, 20]$, indicating the signal sits on a broad plateau rather than at a tuned peak.

Ranking procedure.

```
rank(I, C, subjectFile, anchor):
  word = lower(anchor) drop trailing digits
  for f in files(I) | C,  f != subjectFile:
    s  = refs[f]
    s += 8    if f in C
    s += 10   if |word|>=4 and word in base(f)
    s += 4    if pkg(f)==pkg(subjectFile)
    s -= 1e5  if isTestFile(f)
  sort by (score, refs, name) desc
```

4.3 Draft-then-refine injection

The harness drops all test files, partitions the rest into a **high-confidence** tier (direct callers and dense dependents) and a **lower-confidence** tier, and constructs a pre-drafted answer from the high-confidence tier. It presents this to the agent not as evidence to weigh but as a *starting answer to refine*: keep the high-confidence files unless the graph tools give a concrete reason to drop one; promote any lower-confidence file verified to be a real dependent; do not drop high-confidence files merely for not having re-traced them personally; do not pad with weakly-connected files. The agent then verifies with the graph tools and commits a final ordered list. Handing the model a clean, test-free draft to *edit* rather than a raw ranked list to *rebuild* is what lets it realize the ranker's precision instead of under-committing.

5 Benchmark

Tasks. We evaluate on Apache Hadoop (12,490 Java files). Each of the fourteen impact tasks is derived from a real merged pull request: the anchor is the PR's actual changed symbol, and the gold set G is the real caller/dependent files touched in that PR. Tasks span a wide difficulty range, from tightly-scoped utilities to god-objects referenced across the tree.

Metric. The agent's committed `dependent_files` list is scored by F_1 against G on file basename, truncated to the top k=20 (Sec. 3).

Integrity safeguards. The benchmark is designed to resist both gaming and over-fitting:

- *Single bounded field.* Only the committed list is scored, not prose—an answer cannot inflate recall by mentioning files in passing.
- *Held-out split.* Three tasks are reserved for development; the remaining tasks are never used for any design or tuning decision, and all headline numbers are reported on them.
- *Deduplication.* Tasks with identical gold sets are collapsed so the effective N is honest.
- *Oracle floor.* The raw static blast radius, scored identically, is a hard floor: no arm may claim a graph win on a task unless it exceeds that floor.

Table 1: Ranking quality on held-out tasks. Oracle = raw static blast radius, scored identically.

	Oracle	**GN ranker**
Held-out mean F_1	0.169	**0.519**
Beats oracle floor	—	**all**

Arms. All arms share the same fixed model, repository, prompt, answer contract, and read-only posture, and differ *only* in how the graph is available—isolating the harness as the single independent variable.

- **Control**: the agent with no code graph (grep/read only).
- **MCP**: the same model with the code graph attached as an MCP tool.
- GRAPHNATIVE: the harness of Sec. 4.

6 Results

6.1 Ranking quality in isolation

We first evaluate the ranker alone, scoring its output directly against G on the held-out tasks, with the raw static blast radius as the oracle floor (Table 1). The ranker lifts held-out F1 from 0.17 to 0.52—roughly tripling the usable precision/recall of the static blast radius—and exceeds the oracle floor on every held-out task. This establishes that the structural signals, not the agent, carry the retrieval gain.

6.2 End-to-end agent comparison

Table 2 reports the full held-out matrix for the three agent arms, with mean output tokens as a cost axis. GRAPHNATIVE attains the highest mean F_1 (0.70 vs. 0.61 and 0.57) while emitting the fewest output tokens (≈46% fewer than Control, ≈19% fewer than MCP): more accurate *and* cheaper.

6.3 Analysis

No arm dominates; the benchmark is discriminative. The win counts are split (2/3/3) and the per-task winner changes with task structure. Control wins where brute-force grep over a clean, small dependency structure suffices (RouterRpcServer, HttpServer2); MCP wins where the one-hop caller graph is clean and complete (AbfsClient, DelegTokenRenewer); GRAPHNATIVE wins where the blast radius is large and the central difficulty is *ranking* (Resource, AbfsOutputStream, ByteArrayManager). A benchmark on which every arm scored near-perfect, or on which one arm swept, would be uninformative; this one separates the methods.

Table 2: End-to-end held-out comparison. F_1 at budget k=20; TOK is mean output tokens. Best per row in bold. GRAPHNATIVE abbreviated **GN**.

Task	Ctrl	MCP	**GN**	Win
Resource	0.18	0.13	**0.58**	GN
Server	0.09	**0.36**	0.36	MCP
RouterRpcServer	**0.84**	0.62	0.76	Ctrl
AbfsClient	0.72	**0.92**	0.82	MCP
HttpServer2	**0.87**	0.72	0.72	Ctrl
AbfsOutputStream	0.67	0.67	**1.00**	GN
ByteArrayManager	0.75	0.86	**1.00**	GN
DelegTokenRenewer	0.47	**0.57**	0.38	MCP
Mean F_1	0.57	0.61	**0.70**	
Mean tok	5532	3695	**2995**	
Wins	2	3	3	

The gain is precision under budget. GRAPHNATIVE's advantage concentrates on large, hub-like anchors where the baselines drown in references. There, ranking the firehose—with test files removed, dense and type-matched dependents floated up, and the result handed over as a draft—places the right files in the top 20, which a recall-oriented dump or an under-committing free exploration does not.

Cost. GRAPHNATIVE is consistently the cheapest arm in output tokens. Because it begins from a precise shortlist rather than exploring, it describes fewer false positives and takes fewer retrieval turns. Lower cost at higher accuracy is the practically important combination.

7 Threats to Validity

Construct: retrieval, not synthesis. These tasks measure code-impact *retrieval*—the deliverable is a ranked list of dependent files, not a code change. Within this construct, the draft-then-refine mechanism is partly self-fulfilling, since the draft *is* a prediction of the answer. Whether the harness advantage transfers to producing *diffs*—scored by test-pass or hunk overlap against real fix commits—is a separate question and the natural next evaluation.

Statistical. Per-task numbers are single runs; agent stochasticity makes individual cells (notably the close DelegTokenRenewer result) noisier than the aggregate. The mean gap is more robust than any single cell, but confidence intervals over repeated runs would strengthen per-task claims.

External. Results are on one large Java repository. The structural signals (reference density, caller adjacency, type-name inheritance, package locality) reflect general dependency-graph and JVM naming conventions, but generalization to other languages and codebases remains to be measured.

8 Conclusion

Treating the dependency graph as a coding agent's primary retrieval surface—queried before the first turn, ranked into a precise test-free shortlist by structural signal, and handed to the agent as a draft to refine—yields an agent that is simultaneously more accurate and cheaper at code-impact analysis than both a tool-free baseline and the same model with the graph attached via MCP, on a held-out, gaming-resistant benchmark of real pull requests. The gains come not from a larger model but from relocating retrieval and ranking out of the model's context and into the harness, where the structure of the code can be exploited deterministically. Extending the evaluation from impact retrieval to verified code synthesis is the clear next step.